Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ПЕТРОЗАВОДСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ПетрГУ)
Физико-технический институт
Кафедра информационно-измерительных систем, электроники и автоматики

Хаухия Александр Владимирович

РАСПОЗНАВАНИЕ ОБРАЗОВ ТАНКОВ НА ВИДЕОПОТОКЕ

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

обучающегося 4 курса очной формы обучения
по направлению подготовки бакалавриата
12.04.01 «Приборостроение»

Допущен (а) к защите:

Заведующий кафедрой,	Научный руководитель:
к.ф.-м.н., доцент	к.ф.-м.н., доцент
Тихомиров Александр Андреевич	Тихомиров Александр Андреевич
_____	_____
«___» _____ 2024 г.	«___» _____ 2024 г.

Петрозаводск
2024

Реферат

Работа содержит 53 страницы, 49 рисунков, 9 листингов, 10 источников.

Ключевые слова: распознавание, нейронная, сеть, датасет, YOLOv5, детекция, свёрточная, аугментация.

В данной работе описывается алгоритм создания датасета, обучение свёрточной нейронной сети с помощью алгоритма YOLOv5 и сравнение работы моделей на разных данных.

Цель работы: создание датасета и обучение свёрточной нейронной сети по алгоритму YOLOv5 для распознавания танков.

Задачи:

1. Сбор изображений танков для разметки.
2. Разметка объектов обучения.
3. Аугментация полученных данных.
4. Обучение нейронной сети.
5. Сравнение качества распознавания объектов в зависимости от количества данных.

В данной работе:

1. Были исследованы тренды современной армии и актуальность искусственного интеллекта в военной сфере.
2. Был описан принцип работы свёрточной нейронной сети и YOLOv5 в частности.
3. Был описан принцип разметки данных и последующей аугментации.
4. Были написаны 9 программ для работы.
5. Был создан уникальный датасет техники. Для игровой индустрии он точно был первый.
6. Был описан полный алгоритм работы с YOLOv5 для обучения модели и последующим распознаванием.
7. Были обучены 6 моделей с использованием разных данных. Проведено сравнение и анализ их работы и метрик.
8. Полученные модели были протестированы на реальных данных и предоставлены в отчёте.

Содержание

Введение 5

1. Анализ литературы 6

1.1 Актуальность использования искусственного интеллекта в военной сфере 6

1.2 Введение в задачу распознавания объектов 6

1.3 YOLOv5 7

1.4 Разметка данных 9

1.5 Аугментация данных 9

2. Создание датасета 13

2.1 Сбор видеоматериала 13

2.2 Получение изображений из видео 13

2.3 Разметка данных 14

2.4 Аугментация 15

3. Обучение модели YOLOv5 22

3.1 Установка YOLOv5 22

3.2 Организация данных 22

3.3 Варианты моделей YOLOv5 23

3.4 Начало обучения 23

3.5 GoogleColab 24

3.6 Итоги обучения 24

4. Результаты обучения и анализ данных 25

4.1 Первая модель 25

4.2 Вторая модель 30

4.3 Третья модель 32

4.4 Четвертая модель 34

4.5 Пятая модель 36

4.6 Шестая модель 38

4.7 Анализ работы моделей 40

5. Работа моделей на реальных данных 42

6. Подведение итогов проделанной работы 47

Список литературы 48

Приложения 49

Введение

В наше время, задача обнаружения, классификации и детекции объектов на фото или видео данных находится на пике популярности и востребованности. В различных военных конфликтах используются сотни дронов каждый день. Часть из этих дронов не нуждаются в пилоте, и они могут выполнять определённые задачи самостоятельно. В большинстве из них используются технология распознавания объектов, поэтому данная дипломная работа является актуальной на сегодняшний день.

Цель работы: создание датасета и обучение свёрточной нейронной сети по алгоритму YOLOv5 для распознавания танков.

Задачи:

1. Сбор изображений танков для разметки.
2. Разметка объектов обучения.
3. Аугментация полученных данных.
4. Обучение нейронной сети.
5. Сравнение качества распознавания объектов в зависимости от количества данных.

Обнаружение объектов на изображениях – одна из классических задач в сфере И.И. В наше время обнаружение объектов используется повсеместно во многих аспектах жизни человека, например:

1. Беспилотные автомобили;
2. Видеонаблюдение;
3. Медицина;
4. Промышленность.

Помимо перечисленных областей, нейросети также используются и в военных целях, например:

1. Разведка и наблюдение;
2. Системы безопасности;
3. Беспилотные дроны и летательные аппараты (БПЛА);
4. Робототехника.

Существует множество алгоритмов обнаружения объектов, но в данной работе был использован алгоритм YOLOv5 для работы со свёрточной нейронной сети.

Перед началом работ был проведён поиск похожих работ в открытом доступе, но решений на YOLOv5 и качественного датасета реальных или виртуальных танков не было обнаружено.

1 Анализ литературы

1.1 Актуальность использования искусственного интеллекта в военной сфере

В Российской Федерации активно спонсируется разработка и подготовка кадров для работы в сфере И. И. Об этом говорится в указе Президента России от 10.10.2019 г. № 490. Военная сфера не стала исключением.

На заводе имени Дегтярева был спроектирован и изготовлен роботизированный комплекс «Нерехта», на котором испытывают и отрабатывают искусственный интеллект.

На форуме «Армия-2023» отечественные производители представили множество дронов и БПЛА, которые используются для различных задач от доставки грузов и разведки, до уничтожения целей в воздухе, на земле и под водой.

Современные боевые действий невозможно представить без использования техники оснащённой искусственным интеллектом, поэтому ответ на вопрос об актуальности положительный.

Подробнее про актуальность в статье [1].

1.2 Введение в задачу распознавания объектов

Распознавание объектов входит в состав технологий глубокого обучения. Задача искусственного зрения заключается в обучении нейронной сети искать, классифицировать объекты на изображении или видео. Описание методов реализации распознавания образов указаны в статье [2].

Свёрточные нейронные сети – самый популярный способ реализации компьютерного зрения. Способ работы сети схож с человеческим зрением. С помощью различных свёрток (фильтров) создаются карты признаков, которые выделяют разные особенности изучаемого объекта: формы, цвета, линии, изгибы. Затем к карте признаков добавляется функция активации, позволяющая сети учиться выделять более сложные объекты. Далее карты признаков уменьшаются с помощью пулинга, данные карт при этом сохраняются. После нескольких повторений свёрток и пулинга, полученные признаки отправляются на вход нейросети, которая в свою очередь решает задачу классификации. Стоит отметить, что в зависимости от задачи все процессы могут отличаться.

Рис. 1 – Пример свёрточной нейронной сети для классификации.

1.3 YOLOv5

Перед началом работы было проведено исследование эффективности алгоритмов нейронных свёрточных сетей, в котором, был выбран самый подходящий алгоритм для обнаружения объектов. Примеры сравнения работы различных алгоритмов в статьях [3][4]. YOLOv5 и другие его версии входят в тройку лучших по разным показателям.

Хотелось бы рассказать о плюсах данного алгоритма. Одним из главных плюсов YOLO является простота установки и работы с ним. У данного алгоритма хорошая эффективность работы. Соотношение скорости обучения к качеству и точности распознавания будущей модели делает её на порядок лучше конкурентов вместе с YOLO рассматривались такие алгоритмы как:

1. SSD (Singe Shot Detection);
2. Faster R-CNN;
3. Tiny YOLO.

YOLOv5 – это алгоритм обучения нейронных сетей для обнаружения объектов. «YOLO» расшифровывается как «You only look once» или «Стоит только раз взглянуть».

YOLO использует архитектуру «One-Stage detector». Её суть заключается в том, что на выходе сеть предсказывает с какой вероятностью в прямоугольнике с координатами (x, y, w, h) объект одного из классов.

Для обучения модели нужно иметь некоторое количество изображений и текстовые файлы к этим изображениям, в которых в строчку записаны 5 чисел: первое число – это класс или label, остальные 4 числа, первые два числа – это координаты x и y центральной точки прямоугольника (box) в котором находится наш объект, вторые два числа – это ширина (w) и высота (h) этого прямоугольника.

Ниже на рис. 2 представлен пример размеченных данных, а на рис. 3 представлены размеченные данные для рис. 2. Изображения взяты с официального сайта проекта на GitHub[5].

Рис. 2 – Пример размеченных данных и их параметры.

Рис. 3 – Текстовый файл с размеченными данными.

1.4 Разметка данных

Разметка данных – это второй этап любого датасета после поиска необходимых изображений. Для разметки данных существуют специальные сайты и приложения. В них с помощью разных инструментов можно создать свой датасет с правильно размеченными данными. На просторах интернета можно найти много сайтов для разметок, но самые популярные из них:

1. Roboflow.com;
2. Makesense.ai;
3. Labelbox;
4. Dataloop.

В данной работе был использован сайт makesense.ai. Из его достоинств стоит отметить удобный интерфейс, который ускоряет процесс разметки.

Для разметки нужно просто обвести объект так, чтобы вся его поверхность находилась внутри прямоугольника. Разметку нужно проводить тщательно и аккуратно, так как это напрямую повлияет на работу модели. Пример процесса разметки и интерфейс сайта makesence.ai показаны на рис. 4.

Рис. 4 – Интерфейс сайта makesense.ai.

1.5 Аугментация данных

Для обучения нейронной сети для классификации или обнаружения объектов на изображении предварительно нужно собрать датасет. Для ускорения этого процесса используют аугментацию. Про аугментацию и работы с данными можно прочитать в следующих статьях [6][7].

Аугментация – это метод расширения датасета путём различных преобразований над изображением. Самые популярные виды аугментации:

1. Отражение изображение по вертикали или горизонтали;
2. Случайная обрезка или по-другому кадрирование;
3. Изменения гаммы или цвета;
4. Случайные повороты;
5. Добавления различных шумов.

Аугментация данных имеет ряд преимуществ для обучения различных моделей:

1. Повышает точность;
2. Повышает производительность;
3. Уменьшает переобучение;
4. Увеличивает количество данных;
5. Ускоряет сбор данных.

Аугментацию можно применить к различным типам данных:

1. Изображения;
2. Числовые данные;
3. Текстовые данные;
4. Аудио данные.

Примеры аугментации из статьи [7] представлены на рис. 5.

Рис. 5 – Примеры аугментации изображения.

В Python существуют библиотеки специально предназначены для аугментации. Самые известные библиотеки:

1. Augmentor;
2. AutoAugmen (DeepAugment);
3. Albumenations;
4. Imago.

Подробнее про выше перечисленные библиотеки рассказано в статье [7].

В нашей работе была использована библиотека OpenCV. Эта библиотека широко используется в задачах компьютерного зрения и обработки изображений.

В YOLOv5 также есть встроенная аугментация данных. Виды аугментации YOLO не пересекаются, а дополняют использованные методы этой работы. Методы из YOLO:

1. Наложение полупрозрачных изображений друг на друга;
2. Склеивание нескольких фотографии в одну под разными углами;
3. Наложение на изображение серого прямоугольника в случайном месте;
4. Наложение на изображение другого изображения из датасета.

Рис. 6 – Встроенная аугментация YOLOv5.

2 Создание датасета

2.1 Сбор видео материала

Первый практический этап в работе был сбор данных.

В начале была попытка найти готовый датасет танков в интернете. Поиски ограничились одним датасетом, состоящий из 100 изображений. В эту сотню помимо танков также входили самолёты, поэтому датасет был небольшой и не подходил для задачи в полной мере. Поэтому следующий этап заключался в создании датасета.

При создании своих данных была возможность скачать сотни изображений из интернета, но это заняло бы гораздо больше времени. В последствии было принято неординарное решение, а именно компьютерные игры. В наше время существует множество игр с реалистичной техникой и графикой, приближённой к реальности.

Выбор остановился на игре «World Of Tanks» от компании «Wargaming». Игра была создана в 2008 году при участии программистов из России и Белоруссии. С момента запуска проекта игра получила более 25 престижных наград, количество аккаунтов в 2018 году превысило 160 миллионов, а самое главное это более 600 единиц существующей, экспериментальной (существовавшей только на чертежах) и прототипов техники из 11 стран мира. Реализм игры позволил использовать её для данной работы.

Для датасета нужно было собрать максимально возможное количество техники, чтобы качество распознавания будущей модели было выше. Пришлось потратить на съемки нужного материала несколько дней. По итогу сбора материала получилось собрать несколько часов игрового процесса на различной технике, под разными углами и с разным расстоянием от камеры до техники, различном ландшафте с различными погодными условиями. Это всё делалось для создания более точной модели.

2.2 Получение изображений из видео

Для того чтобы получить отдельные кадры из видеоряда была написана программа на языке Python (Приложение А).

Как было сказано ранее в работе использовалась библиотека OpenCV. Из библиотеки был взят класс VideoCapture и его методы get, read, isOpened и imwrite.

Метод get используется для получения значения кадров в секунду (fps).

Метод read используется для получения, декодирования и использования кадра из видео.

Метод isOpened используется для цикла while. До конца видео для каждого кадра этот метод возвращает True, а в конце видео цикл заканчивается.

Метод imwrite позволяет сохранить кадр.

После получения кадров из видео их нужно было отсортировать, а именно удалить кадры где нет техники, идентичные кадры и кадры с плохим качеством.

Как было сказано ранее все этапы выполнялись дважды и в первый раз были получены 376 уникальных изображений, во второй раз 496.

2.3 Разметка данных

Следующим этапом работы была разметка танков на подготовленных изображениях. При разметке необходимо отметить нужный нам объект, в нашем случае танк, и определить размеченный объект к какому-либо классу. К этому процессу нужно подходить ответственно, для того чтобы свёрточная нейронная сеть не накапливала в карте признаков лишнее.

Главный вопрос этого процесса: какие данные являются валидными? К валидным данным в этой работе относятся:

1. Танки на открытой местности;
2. Части танков, которые видно из-за рельефа, строений и растительности.

К невалидным данным относятся:

1. Уничтоженные танки;
2. Танки вне игрового процесса.

Как было сказано ранее мы использовали онлайн ресурс makesense. Алгоритм работы с ним следующий:

1. Загрузить изображения;
2. Загрузить файлом или вручную ввести классы (labesls);
3. Разметить данные и отнести их к нужному классу;
4. Выгрузить архив с разметками (.txtфайлы).

Итогами разметки являются изображения из пункта 2.2 и текстовые файлы после разметки. Можно сказать, что это уже готовый датасет за одним исключением, количество текстовых файлов отличается от количества изображений. Это связанно с ошибками при сортировке данных, а именно остаются некачественные или идентичные фотографии, которые не используются для разметки. Удаление изображений необязательный процесс, но он уменьшает общий размер будущих архивов и делает их более удобными для чтения и работы с ними.

Для удаления была написана небольшая программа (Приложение Б).

2.4 Аугментация

Чтобы качественно обучить модель, нужно собрать как можно больше данных, в нашем случае это изображения. Так как процессы сбора, сортировки и разметки данных долгие, было принято решение воспользоваться аугментацией данных. Для работы выбраны следующие виды:

1. Отражение изображений по горизонтали и вертикали;
2. Изменение цветов из RGB (красный, зелёный, синий) в BRG и чёрно-белый;
3. Добавление шума на изображение.

Ниже будет подробное описание работы всех программ. Исходное изображение для всех программ на рис. 7.

Рис. 7 – Исходное изображение.

Первые две программы – это отражение изображения по горизонтали и по вертикали (Приложение В, Г). Для этого были использованы две библиотеки: OS – это библиотека для работы с файлами и папками, PIL (Python Imaging Library) – это библиотека для работы с изображениями.

Для начала необходимо создать два объекта, которые хранят в себе путь к исходным изображениям и путь в новую папку, где будут храниться полученные изображения.

С помощью функции listdir мы получаем список файлов, которые находятся в папке с нашими изображениями.

С помощью функции makedirs мы создаём папку для отражённых изображений, если она не существует.

Далее для каждого изображения в папке input_folder мы делаем следующие действия:

1. С помощью функции join создаём путь для входного и выходного изображения;
2. Открываем изображение;
3. С помощью функции transpose и FLIP_LEFT_RIGHT отражаем изображение меня левую и правую сторону изображения;
4. И сохраняем изображение в новую папку с припиской «h» в начале имени входного изображения.

Итогом этой программы является новая папка с отражёнными изображениями. Пример на рис. 8.

Рис. 8 – Отражённое по горизонтали изображение.

Для отражения изображения по вертикали сохраняется аналогичный алгоритм за исключением нескольких пунктов:

1. Приписка в выходных изображениях меняется с «h» на «v»
2. Вместо функции FLIP_LEFT_RIGHT используется функция FLIP_TOP_BOTTOM.

Результат работы на рис. 9.

Рис. 9 – Отражённое по вертикали изображение.

Как говорилось в пункте 1.3, формат размеченных данных – это 5 чисел: класс, оси X и Y центральной точки прямоугольника, а также ширина и высота этого прямоугольника. Так как мы отразили изображения, положения центральных точек изменились и для того, чтобы разметка осталась на своём месте, были написаны следующие программы (Приложение Д, Е).

В этих программах по аналогии из предыдущих программ используется библиотека OS, создаётся два объекта, в которых храниться путь к разметке и путь в новую папку, создаётся список из всех элементов в папке, создаётся путь к входным и выходным данным. Далее программы отличаются.

Мы открываем текстовый файл и для каждой строчки в этом файле производим следующее действия:

1. Разбиваем строку на числа и сохраняем в список;
2. Вычитаем из единицы второе число в списке для горизонтального отражения или третье число для вертикального отражения (см. рис. 2);
3. Округляем до 6 знаков после запятой (стандарт для YOLO);
4. Записываем изменённый список в новый файл.

Итогом программы является папка с изменёнными текстовыми файлами.

Благодаря этим четырём программам мы увеличили свой датасет в три раза, а значит точность распознавания модели выше. Важно отметить, что отражённые изображения хранят в себе признаки отличные от оригинальных и не влияют на переобучение модели.

Рис. 10 – Исходные и изменённые данные.

Следующие две программы из приложений Ж и И изменяют палитру цветов на изображении. В этих двух программах была использована библиотека OpenCV и функция cvtColor. Алгоритм программ похож на предыдущие.

Итоги программ на рис. 11 и рис. 12.

Рис. 11 – Изображение после изменение цвета с RGBна BRG.

Рис. 12 – Изображение с чёрно-белым фильтром.

Последняя программа для аугментации данных – это добавление шума на изображение (Приложение К). Подробнее про виды шума и способы добавления их на изображение в статьях[8][9].

В этой программе была использована библиотека numpy (Numerical Python) – это библиотека с различными математическими функциями, которая позволяет работать с матрицами, создавать графики, анализировать данные и проводить различные математические вычисления.

В данной программе numpy используется для создания случайного шума. Для настройки шума используются две переменные mean и std.

mean – среднее значение шума. Если это число равно нулю, то средний шум будет черный (RGB (0,0,0)).

std – стандартное отклонение шума. Чем больше это число, тем более размытый шум на изображении.

Например, если mean равен 0, а std равен 10, то шум будет варьироваться у нуля с отклонением 10.

Возвращаясь к программе, в цикле сначала создаётся шум с помощью модуля random, а затем этот шум с помощью функции add добавляется на изображение.

Итог программы на рис. 13.

Рис. 13 – Зашумленное изображение.

В отличии от программ, которые отражают изображения, программы для изменения цветовой палитры или добавления шума не нуждаются в изменении разметки.

Благодаря написанным программам датасет был увеличен в 6 раз. Программы можно комбинировать разными способами и увеличить датасет больше чем в 6 раз. Общий объем изображений в датасете составил 5232 изображений и столько же разметок. Из них:

1. 376 фотографий из первого сбора данных;
2. 1880 фотографий из первого сбора данных аугментированных;
3. 496 фотографий из второго сбора данных;
4. 2480 фотографий из второго сбора данных аугментированных.

3 Обучение модели YOLOv5

3.1 Установка YOLOv5

Первый этап в обучении – установка нужных пакетов, библиотек и файлов для работы с YOLOv5. Для этого нужно в консоли написать несколько строчек кода (рис. 14).

```
git clone https://github.com/ultralytics/yolov5  # clone
cd yolov5
pip install -r requirements.txt  # install
```

Рис. 14 – Код для установки YOLO и библиотек для работы с ним.

После выполнения кода у нас будут установлены необходимые файлы и библиотеки для работы. В этой сборке YOLO умеет детектировать 80 различных объектов, но наша задача обучить модель на нашем датасете.

3.2 Организация данных

Для подготовки собственного датасета мы разделяем наши данные на тренировочные, валидационные и тестовые. В нашей работе мы не использовали тестовые данные, а в роли тестовой выборки использовались валидационные данные. Валидационные данные нужны для проверки работы модели и помогают избежать переобучение.

В интернете можно найти различные советы по разделению данных для обучения, например, 60 процентов тренировочные, 20 процентов валидационные и 20 процентов тестовые. В нашей работе мы использовали 80 процентов тренировочных и 20 процентов валидационных.

Рис. 15 – Разделение датасета. [10]

После разделения данных нужно создать файл с расширением .yaml, в котором мы указываем путь до выборок и классы для распознавания объектов.

Рис. 16 – Листинг файла с расширением .yaml.

3.3 Варианты моделей YOLOv5

В YOLOv5 есть несколько вариантов готовых моделей рис. 17 [10]. Они отличаются скоростью и качеством работы итоговой сети. Мы использовали YOLOv5s, так как мощность имеющийся видеокарты не позволяла выбрать более мощную модель.

Рис. 17 – Варианты моделей YOLOv5.

3.4 Начало обучения

С помощью команды на рис. 18 начинается обучение модели. В ней мы можем настроить некоторые параметры обучения. В данной работе были использованы: кэширование данных, выбор модели, настройка размеров изображений и количество эпох. Кэширование данных позволяет ускорить обучение Увеличение размера изображений замедляет обучение, но увеличивает качество распознавания.

```
python train.py --img 1088 --epochs 100 --data dataset.yaml --weights yolov5s.pt --cache
```

Рис. 18 – Команда для начала обучения.

3.5 GoogleColab

Обучать модель можно с помощью сервиса GoogleColab. У бесплатной версии есть свои плюсы и минусы.

Плюсы:

1. Простота установки библиотек;
2. Скорость обучения;
3. Нагрузка идёт на видеопамять удалённой машины.

Минусы:

1. Время работы 4 часа;
2. Медленная загрузка данных на сервер;
3. Ограничение на загружаемый объем данных.

Со всеми минусами мы столкнулись в процессе работы, что замедлило выполнение работы.

3.6 Итоги обучения

Итогами обучения являются веса, которые можно использовать для распознавания объектов, графики обучения, примеры тренировочных и валидационных выборок, метрики обучения для каждой эпохи.

4 Результаты обучения и анализ данных

Для обучения модели и сравнения качества распознавания все данные были разделены на следующие датасеты:

1. Первый сбор данных без аугментации (376 изображения);
2. Первый сбор данных с аугментацией (2256 изображения);
3. Второй сбор данных без аугментации (496 изображения);
4. Второй сбор данных с аугментацией (2976 изображения);
5. Первый и второй сбор данных без аугментации (872 изображения);
6. Первый и второй сбор данных с аугментации (5232 изображения).

Далее будут предоставлены сравнения моделей, обученных на разных объемах данных.

4.1 Первая модель

Первая модель состоит из самого маленького набора изображений. В неё больше всего недочётов и ошибок:

1. Не все объекты были размечены;
2. Качество разметки не самое лучшее;
3. Разнообразие техники и ландшафтов низкое;
4. Игровая информация не была убрана и создавала помехи и шумы.

Во втором сборе данных все эти недочеты были устранены.

Длительность первого обучения составила 0,544 часа. Обучение проходило на видеокарте NVIDIA GeForce RTX 3060 с 12 Гб видеопамяти из которых в среднем использовалось 10,5 Гб.

Ниже будут подробно описаны все результаты и предоставлены изображения распознавания.

На рис. 19 изображён график F1-меры. Она используется для оценки качества распознавания модели. Формула F1-меры (1):

$$F1 = \frac{2}{\frac{1}{\text{Полнота}} + \frac{1}{\text{Точность}}}, \quad (1)$$

где точность – это отношение правильных предсказаний ко всем сделанным предсказаниям, а полнота – это отношение правильных предсказаний среди всех фактических предсказаний.

Пороговое значение уверенности – это значение при котором модель решает, нужно ли

классифицировать объект или нет. Чем больше пороговое значение уверенности, тем меньше модель делает ложных предсказаний, но больше правильных предсказаний пропускает.

F1-мера является средним взвешенным между точностью и полнотой, поэтому она считается наиболее сбалансированной мерой производительности модели.

Максимальное значение F1-меры на рис. 19 равно 0,66, это означает, что модель имеет точность и полноту равную 0,66 при пороговом значении уверенности 0,336.

Рис. 19 – График F1-меры первой модели.

На рис. 20 изображены графики разных параметров на всех эпохах обучения. Как было сказано ранее эпох было 100.

График box_loss показывает процент ошибок в изображении сетки над объектом. График obj_loss показывает ошибки в определении наличия объекта внутри сетки. Видно, что obj_loss растёт. Это означает, что модель переобучена и в четвёртой модели мы подробнее разберём этот аспект.

График precision показывает точность модели. График recall показывает полноту модели.

Самые интересные графики – это mAP_0.5 и mAP_0.5:0.95. Они показывают достоверность классификации с качеством 0,5 и с качеством 0,95 соответственно. В нашем случае максимальная достоверность mAP_0.5 равна 0,656, а mAP_0.5:0.95 равна 0,319.

Рис. 20 – Графики метрик первого обучения.

В результатах обучения также есть графики точности от уверенности, точности от полноты, полноты от уверенности.

На рис. 21 предоставлен пример обучающей партии. В пункте 1.5 говорилось о встроенной аугментации, на рис. 21 это отлично видно. Некоторые тренировочные изображения состоят из нескольких изображений из нашей выборки. Также используется поворот изображения. Можно обратить внимание на ошибки при разметке, так на 3391.png ни один из танков не отмечен.

Рис. 21 – Пример обучающей партии.

На рис. 22 показаны размеченные нами объекты, входящие в валидационные данные (в нашем случае тестовые), а на рис. 23 показано как эти же объекты распознала модель. Можно заметить, что модель распознала часть не отмеченных нами танков, но также не распознала другие танки, а процент уверенности низкий. На этих рисунках примера нет, но модель также определяет некоторые дома, кусты и пригорки как танки

Рис. 22 – Размеченные нами валидационные данные.

Рис. 23 – Размеченные моделью валидационные данные.

На рис. 24 показана команда для начала детекции.

Рис. 24 – Команда для начала распознавания.

На рис. 25 показан результат работы модели. Все танки были обнаружены, средняя уверенность приблизительно 0,7.

Рис. 25 – Результат распознавания первой модели.

4.2 Вторая модель

Вторая модель была обучена на первых данных с использованием аугментации. Длительность обучения составила 2,5 часа.

Максимальное значение F1-меры на рис. 26 равно 0,77.

Рис. 26 – График F1-меры второй модели.

Значение mAP_0.5 равно 0,792, а mAP_0.5:0.95 равно 0,383. Эти значения выше чем у первой модели и из этого можно сделать вывод, что аугментация, в данном случае, положительно сказалась на результатах метрик.

Рис. 27 – Графики метрик второго обучения.

На рис. 28 предоставлен результат работы. На этом изображении значения уверенности хуже, но в целом результат лучше.

Рис. 28 – Результат распознавания второй модели.

4.3 Третья модель

Третья модель похожа на первую, но в ней были исправлены недочёты. Длительность обучения составила 0,591 часа.

Максимальное значение F1-меры на рис. 29 равно 0,88.

Рис. 29 – График F1-меры третьей модели.

Значение mAP_0.5 равно 0,912, а mAP_0.5:0.95 равно 0,522.

Рис. 30 – Графики метрик третьего обучения.

На рис. 31 предоставлен результат работы. Все танки были обнаружены. Уверенность модели варьируется от 0,82 до 0,93. Эти значения выше чем у первой модели и второй модели.

Рис. 31 – Результат распознавания третьей модели.

4.4 Четвертая модель

Четвёртая модель состоит из данных второго сбора с использованием аугментации. Длительность обучения составила 5,7 часа.

Максимальное значение F1-меры на рис. 32 равно 0,85.

Рис. 32 – График F1-меры четвёртой модели.

Значение mAP_0.5 равно 0,882, а mAP_0.5:0.95 равно 0,502. Эти значения также хуже, чем у третьей модели, но модель лучше детектирует зашумленные объекты.

Рис. 33 – Графики метрик четвёртого обучения.

На рис. 34 предоставлен результат работы. Все танки были обнаружены. Уверенность модели высокая и составляет примерно 0,9. Также модель хорошо детектирует объекты на удалении от камеры. Забегая наперёд, данная модель оказалась лучшей.

Рис. 34 – Результат распознавания четвёртой модели.

4.5 Пятая модель

Пятая модель обучалась на сумме данных из первой и третьей выборки. Длительность обучения составила 0,983 часа.

Максимальное значение F1-меры на рис. 35 равно 0,89.

Рис. 35 – График F1-меры пятой модели.

Значение mAP_0.5 равно 0,906, а mAP_0.5:0.95 равно 0,491. Эти значения хуже, чем у третьей модели, но не значительно.

Рис. 36 – Графики метрик пятого обучения.

На рис. 37 предоставлен результат работы. Все танки были обнаружены. Уверенность модели ниже, чем у третьей. Она варьируется в диапазоне от 0,71 до 0,85.

Рис. 37 – Результат распознавания пятой модели.

4.6 Шестая модель

Шестая модель обучалась на всех данных с использованием аугментации. Длительность обучения составила более 13 часов.

Максимальное значение F1-меры на рис. 38 равно 0,84. Это значение ниже, чем в пятой модели.

Рис. 38 – График F1-меры шестой модели.

Значение mAP_0.5 равно 0,869, а mAP_0.5:0.95 равно 0,463.

Рис. 39 – Графики метрик шестого обучения.

На рис. 40 предоставлен результат работы. Уверенность модели высокая и составляет больше 0,9 во всех случаях. Также модель хорошо детектирует зашумленные объекты, но хуже детектирует объекты на расстоянии.

Рис. 40 – Результат распознавания шестой модели.

4.7 Анализ работы моделей

В этом пункте будет анализ работы моделей.

Первое, что хотелось бы проанализировать – это метрики. Метрики – один из главных

параметров модели в отличии от оценки работы над изображениями. На таблице 1 предоставлены все главные параметры. Лучшие метрики у третьей модели и кажется, что она будет лучше работать, но это не так.

Модели / Метрики	Первая	Вторая	Третья	Четвёртая	Пятая	Шестая
F1-мера	0,66	0,77	0,88	0,85	0,89	0,84
Precision(точность)	0,589	0,816	0,924	0,926	0,923	0,878
Recall (полнота)	0,754	0,729	0,842	0,782	0,864	0,801
mAP_0.5	0,656	0,792	0,912	0,88	0,906	0,869
mAP_0.5:0.95	0,319	0,383	0,522	0,501	0,491	0,463

Таблица 1 – Метрики моделей.

Стоит отметить, что модели, которые обучались на данных без аугментации, решают одну задачу: распознавание танков на чистых изображениях, а модели, которые обучались с использованием аугментации, более универсальные и эти модели решают уже несколько задач: распознавание танков на чистых, шумных и искаженных каким-либо способом изображениях. Поэтому на практике модели с аугментацией будут работать лучше, чем модели без неё.

Следующий аспект оценки моделей – это оценка изображений. Оценка изображений субъективный параметр. На каждое изображение можно найти модель, которая будет лучше распознавать объекты. Пример на рис. 41. На рис. 41 видно, что на левом изображении лучше качество у шестой модели, а на правом изображении лучше качество у четвёртой модели. Поэтому оценивать модели по одному изображению – неверный подход, лучше смотреть видео с работой модели.

Рис. 41 – Сравнение четвёртой (сверху) и шестой (снизу) моделей.

5 Работа моделей на реальных данных

Последнее что было сделано в данной работе это проверка работы моделей на реальных данных. Были взяты две лучшие модели и видеоролики из интернета на которых с помощью дрона записана техника. Для демонстрации работы были выбраны 4 видеоряда. Ниже предоставлены изображения работы моделей четыре и шесть.

Изображения были выбраны из тех, которые лучше всего распознались. Модели работают неидеально, часть техники не распознаётся, иногда пропадает выделение, зачастую незначительные помехи сильно влияет на качество работы модели. На рис. 46 модель выделила грузовик с уверенностью в 0,56. Также стоит отметить, что качество распознавания лучше, когда объект снимается сверху.

Если учитывать тот факт, что модели обучены на данных из компьютерной игры и наш датасет довольно мал, если сравнивать его с датасетами других объектов, качество работы модели превысило наши ожидания.

Рис. 42 – Результат работы четвёртой модели.

Рис. 43 – Результат работы шестой модели.

Рис. 44 – Результат работы четвёртой модели.

Рис. 45 – Результат работы шестой модели.

Рис. 46 – Результат работы четвёртой модели.

Рис. 47 – Результат работы шестой модели.

Рис. 48 – Результат работы четвёртой модели.

Рис. 49 – Результат работы шестой модели.

6 Подведение итогов проделанной работы

Подводя итоги проделанной работы хотелось бы начать с самого начала.

1. Были исследованы тренды современной армии и актуальность искусственного интеллекта в военной сфере.
2. Был описан принцип работы свёрточной нейронной сети и YOLOv5 в частности.
3. Был описан принцип разметки данных и последующей аугментации.
4. Были написаны 9 программ для работы.
5. Был создан уникальный датасет техники. Для игровой индустрии он точно был первый.
6. Был описан полный алгоритм работы с YOLOv5 для обучения модели и последующим распознаванием.

7. Были обучены 6 моделей с использованием разных данных. Проведено сравнение и анализ их работы и метрик.
8. Полученные модели были протестированы на реальных данных и предоставлены в отчёте.

Данную работу можно развивать и улучшать дальше. Можно увеличить датасет, добавить в него реальные изображения техники, улучшить и добавить новые алгоритмы аугментации данных, загрузить модель на миникомпьютер rasberypie и использовать её на дроне.

Планируется продолжать проект в будущем, в магистратуре.

Список литературы

1. Макаренко С. И. Робототехнические комплексы военного назначения – современное состояние и перспективы развития [Электронный ресурс] / Макаренко С. И. // Системы управления, связи и безопасности. - Санкт-Петербург, 2016. – Режим доступа: https://cyberleninka.ru/article/n/robototehnicheskie-kompleksy-voennogo-naznacheniya-sovremennoe-sostoyanie-i-perspektivy-razvitiya/viewer
2. Методы распознавания образов: от простых до сложных: [Электронный ресурс] // simbirsoft.com. 2023. – URL: https://www.simbirsoft.com/blog/metody-raspoznavaniya-obrazov-ot-prostykh-do-slozhnykh/ . (дата обращения 07.05.2024)
3. Выбираем быстрый детектор людей в 2021: [Электронный ресурс] // habr.com. 2021. – URL: https://habr.com/ru/articles/598191/ . (дата обращения 14.05.2024)
4. Comparative analysis of deep learning image detection algorithms / S. Srivastava, A. V. Divekar, C. Anilkumar, I. Naik, V. Kulkarni, V. Pattabiraman. // Journal of Big Data 8. – 2021. – Режим доступа: https://doi.org/10.1186/s40537-021-00434-w
5. Train Custom Data – Ultralytics YOLO Docs: [Электронный ресурс] // docs.ultralytics.com. 2024 – URL: https://docs.ultralytics.com/yolov5/tutorials/train_custom_data/ . (дата обращения 26.04.2024)
6. Data Augmentation in Python: Everything You Need to Know: [Электронный ресурс] / V. Lyashenko // The MLOps Blog. – 2024. – Режим доступа: https://neptune.ai/blog/data-augmentation-in-python . (дата обращения 26.04.2024)
7. Аугментация изображений нейронные сети [Электронный ресурс] / triptonkosti.ru. 2024. – URL: https://triptonkosti.ru/21-foto/augmentaciya-izobrazhenij-nejronnye-seti.html . (дата обращения 14.05.2024)
8. Guide to Adding Noise Images with Python and OpenCV [Электронный ресурс] // askpython.com. 2023. – URL: https://www.askpython.com/python/examples/adding-noise-images-opencv . (дата обращения 01.05.2024)
9. Correctly add Gaussian noise to image-python [Электронный ресурс] // programmersought.com. 2018. – URL: https://programmersought.com/article/81676398528/ . (дата обращения 01.05.2024)
10. Validation Data в Машинном обучении простыми словами [Электронный ресурс] // dzen.ru. 2021. – URL: https://dzen.ru/a/YD_Lbq_ADkxlh0QD . (дата обращения 22.05.2024)

ПРИЛОЖЕНИЯ

Приложение А.

Рис. 1 – Листинг программы для сохранения кадров из видео.

Приложение Б.

Рис. 2 – Листинг программы для удаления лишних фотографий.

Приложение В.

Рис. 3 – Листинг программы для отражения изображения по горизонтали.

Приложение Г.

Рис. 4 – Листинг программы для отражения изображения по вертикали.

Приложение Д.

Рис. 5 – Листинг программы для изменения разметки после отражения изображения по горизонтали.

Приложение Е.

Рис. 6 – Листинг программы для изменения разметки после отражения изображения по вертикали.

Приложение Ж.

Рис. 7 – Листинг программы для изменения цветовой гаммы изображения с RGB на BRG.

Приложение И.

Рис. 8 – Листинг программы для изменения цветовой гаммы изображения с RGB на чёрно-белый.

Приложение К.

Рис. 9 – Листинг программы для добавления шума на изображение.